

Grado en Ingeniería del Software
Doble Grado en Matemática Computacional e Ingeniería del Software
Doble Grado en Física Computacional e Ingeniería de Software

Bases de Datos



Práctica 1 Modelo Relacional y Lenguaje SQL Avanzado

Rafael Socas Gutiérrez
Amador Maho Etohá
Ana Isabel Sánchez González

Curso 2024/25



Datos de los Alumnos

#	Nombre y Apellidos	Curso
1	Diego Jaime Vega Sánchez	2
2	Matías Svriz	2
3	Víctor Manuel Peiró Martínez	2

Instrucciones

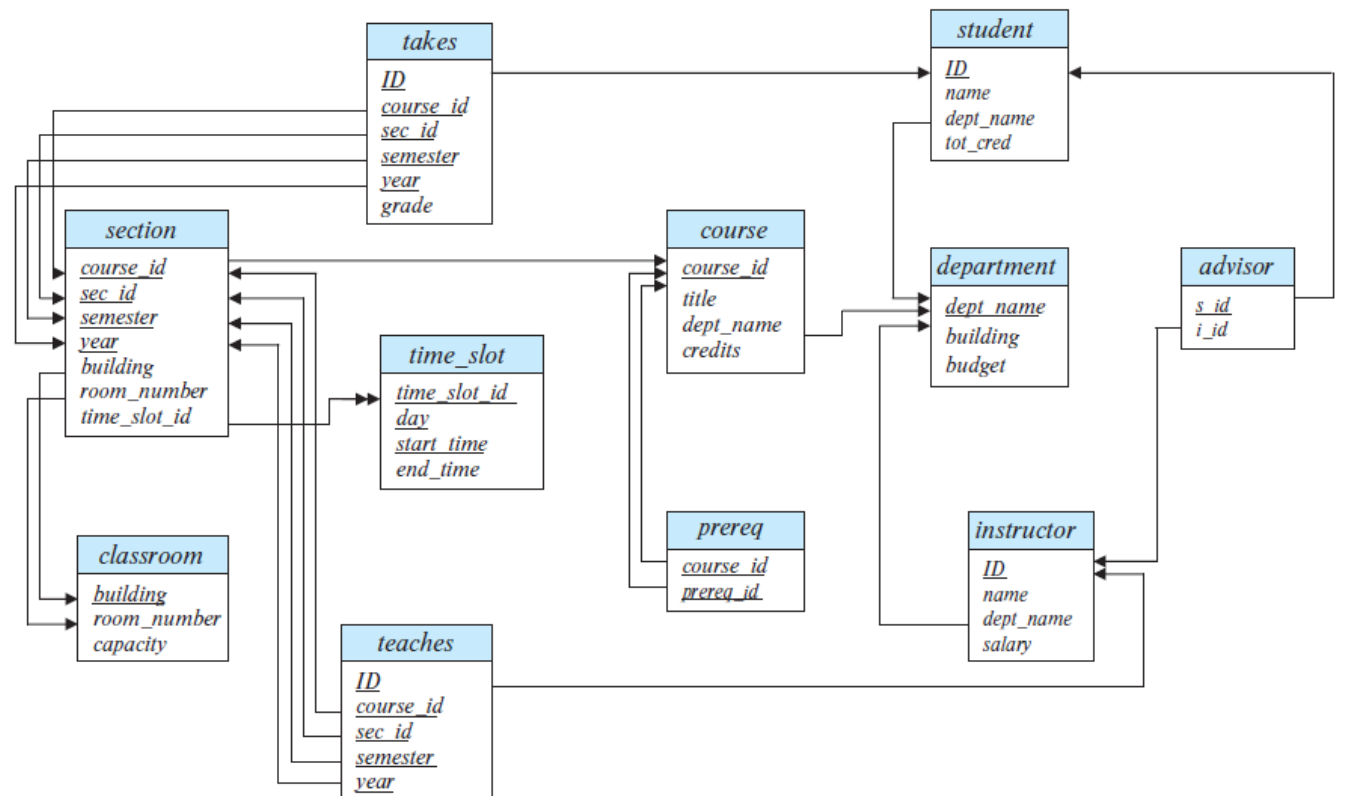
- Completa la práctica en este mismo Power Point rellenando las páginas en blanco o incluyendo más páginas si necesitas más espacio para los pantallazos y las explicaciones.
- Una vez completado el Power Point, guárdalo en formato pdf. **A la plataforma BB sube el pdf resultante.**
- **IMPORTANTE.** Adjunta también un script con todas las queries de Algebra Relacional y SQL que has utilizado para resolver las diferentes preguntas.
- Rellene el nombre/apellidos y el curso de los participantes del grupo.
- **Fecha máxima de entrega: Domingo 23 de Marzo del 2025 a las 23:59**

Enunciado

Se dispone de la BBDD **university** de la Universidad de Yale (https://www.db-book.com/university-lab-dir/sample_tables-dir/index.html) cuyo **Esquema** se presenta a continuación:

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

Cargue esta BBDD en MySQL usando los ficheros **DDL.sql** y **smallRelationsInsertFile.sql** que se adjuntan

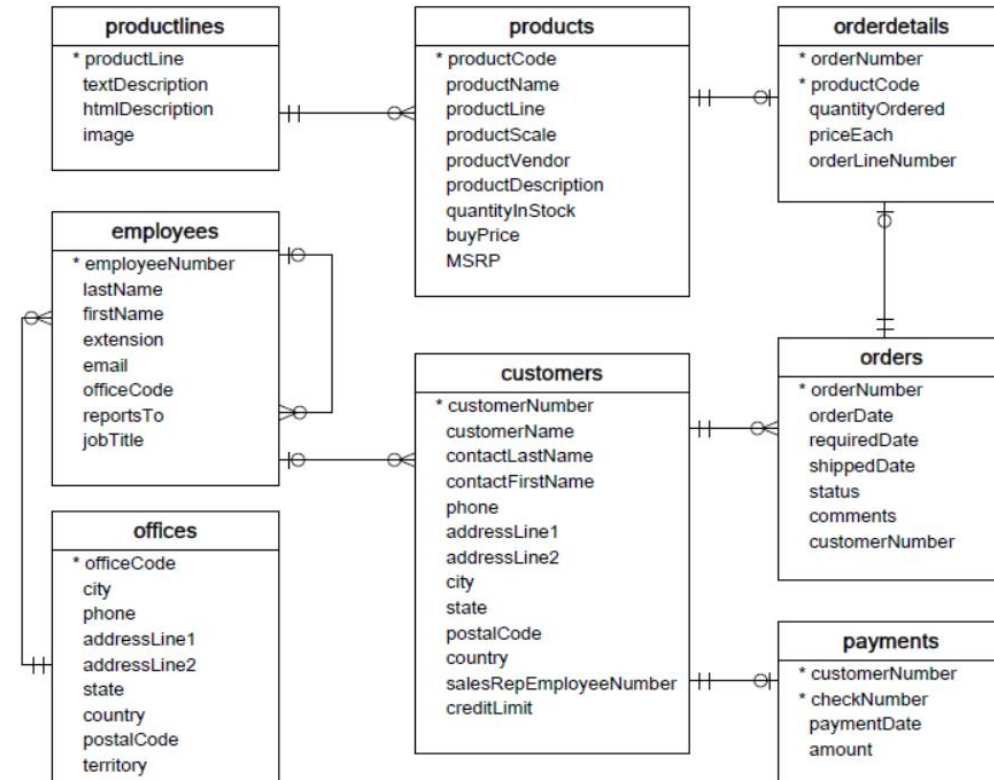


Enunciado

Se dispone también de la BBDD **classicmodels** que es la BBDD de ejemplo del tutorial de MySQL (<https://www.mysqltutorial.org/mysql-sample-database.aspx>) cuyo **Esquema** se presenta a continuación:

- **Customers:** stores customer's data.
- **Products:** stores a list of scale model cars.
- **ProductLines:** stores a list of product line categories.
- **Orders:** stores sales orders placed by customers.
- **OrderDetails:** stores sales order line items for each sales order.
- **Payments:** stores payments made by customers based on their accounts.
- **Employees:** stores all employee information as well as the organization structure such as who reports to whom.
- **Offices:** stores sales office data.

Cargue esta BBDD en MySQL usando el fichero `mysqlsampledatabse.sql` que se adjunta.



1) **PREGUNTA (1 punto):** dada la tabla *office*, escriba en álgebra relacional y presente la tabla resultante de cada de los siguientes apartados:

1. El código para presentar solamente las tuplas donde el territorio sea EMEA y la ciudad no sea London.
2. Sobre la consulta anterior, escribe el código en álgebra relacional para obtener una relación donde solo estén: el nombre de la ciudad (city), el país (country) y el officeCode.
3. Renombra la relación del ejercicio anterior como "EUoffice" ("oficina de la Unión Europea").

office

officeCode	city	phone	addressLine1	country	postalCode	territory
1	San_Francisco	16502194782	100 Market Street	USA	94080	NA
2	Boston	12158370825	1550 Court Place	USA	2107	NA
3	NYC	12125553000	523 East 53rd Street	USA	10022	NA
4	Paris	33147234404	43 Rue Jouffroy D'abbans	France	75017	EMEA
5	Tokyo	81332245000	4-1 Kioicho	Japan	1028578	Japan
6	Sydney	6129264451	5-11 Wentworth Avenue	Australia	2010	APAC
7	London	442078772041	25 Old Broad Street	UK	175	EMEA

1. `SELECT * FROM offices WHERE territory = 'EMEA' AND city != 'London'`

De la tabla `offices` mostramos todas las columnas cuyas tuplas verifiquen las condiciones de "WHERE" y "AND", en este caso solo una

```
mysql> select * from offices where territory = 'EMEA' AND city != 'London';
```

officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
4	Paris	+33 14 723 4404	43 Rue Jouffroy D'abbans	NULL	NULL	France	75017	EMEA

2. $\pi_{\text{city, country, officecode}} (\sigma_{\text{territory}='EMEA' \wedge \text{city} \neq \text{'London'}}(\text{offices}))$

Seleccionamos las columnas pertinentes de "offices" y mostramos las tuplas con la condición previa

3. `EUOffice` $\leftarrow \pi_{\text{city, country, officecode}} (\sigma_{\text{territory}='EMEA' \wedge \text{city} \neq \text{'London'}}(\text{offices}))$

Añadimos el alias `EUOffice` a la instrucción

2) PREGUNTA (1 punto): indica e interpreta lo que se obtiene después de ejecutar las siguientes expresiones de Álgebra relacional sobre las dos relaciones dadas :

Estudiante (dni_est, nomb_est)

Profesor (dni_prof, nomb_prof).

dni_est	nomb_est
12345	Perez
968574	Valdez
192837	Juarez

dni_prof	nomb_prof
654789	Perez
852741	Fernandez
192837	Juarez

1. Estudiante $\cup$ Profesor
2. Estudiante $\cap$ Profesor
3. Profesor $-$ Estudiante

¿De cuántas tuplas consta cada relación resultante?

¿Qué condición deben de cumplir ambas relaciones Estudiante y Profesor entre sí para poder realizarse las operaciones de álgebra relacional indicadas?

Para poder usar las relaciones de algebra relacional, se han de cumplir 2 condiciones:

1. Las Relaciones han de tener el mismo numero de atributos
2. El dominio de ambas relaciones ha de ser el mismo. Es decir que han de tratar con los mismos tipos de datos

Ambas condiciones se cumplen asi que podemos hacer esta operaciones

1. Estudiante U Profesor. Esto coge todos los campos ya que es la union de las relaciones. Como hay una repetida, se devolveran las siguientes 5 tuplas:

dni	nombre
12345	Perez
968574	Valdez
192837	Juarez
654789	Perez
852741	Fernandez

2. Estudiante $\cap$ Profesor. Esto solo coge las tuplas que estan en ambas relaciones ya que es una intersección. Por ello, solo devolvera una tupla que es la siguiente

dni	nombre
192837	Juarez

3. Profesor — Estudiante. Esto coge las filas que estan en profesor y no en estudiante. Por ello, solo se devuelven las siguientes 2 tuplas:

dni	nombre
654789	Perez
852741	Fernandez

3) PREGUNTA (1 punto): de la base de datos **university**, se quiere obtener en una sola consulta, el nombre del departamento (*dept_name*) que tiene el mayor número de profesores.

```
SELECT dept_name
FROM instructor i
GROUP BY dept_name
ORDER BY COUNT(i.ID) DESC
LIMIT 1;
```

Con "LIMIT 1;" y "DESC" hacemos que una vez calculado el número de coincidencias en dept_name en todas las tuplas, se muestra unicamente la que aparezca más veces. Con esta lógica tendrás el número de instructores en cada departamento y mostrarás que departamento tiene más.

	Az dept_name ▼	
1	Comp. Sci.	🔗

4) PREGUNTA (1 punto): responda la siguiente pregunta: ¿Cuál es la clase dónde se impartieron más asignaturas en el año 2018? (BBDD university)

Condiciones: para responder la pregunta debes conseguir en una sola consulta, mostrar una tabla con la información solicitada.

```
SELECT s.building, s.room_number
FROM section s
WHERE s.year = 2018
GROUP BY s.building, s.room_number
ORDER BY COUNT(s.course_id) DESC
LIMIT 1;
```

Siguiendo la misma lógica que en el punto anterior calculamos cuantos cursos han sido asignados a una clase, por ello agrupamos por clase y contamos en función del ID del curso. Como se pide unicamente el año de 2018 se añade la condición del "Where".

	A-z building	A-z room_number	
1	Packard	101	

5) PREGUNTA (1 punto): en la base de datos **university**, dentro de la tabla *course* hay un curso que no está vinculado a ninguna oferta o *section* y queremos asignarlo al aula menos utilizada del año 2018 para el siguiente año. En una sola query muestra una tabla donde se vea el curso que no está en ninguna *section*. ¿Cuál es el nombre de la asignatura? ¿A qué departamento pertenece este curso? ¿Y cuál es el aula menos utilizada del año 2018?

```
select c.course_id , c.title as course_name, c.dept_name
from course c left join section s on s.course_id = c.course_id
where s.course_id is null;
```

	A-Z course_id	A-Z course_name	A-Z dept_name	
1	BIO-399	Computational Biology	Biology	

```
SELECT s.building, s.room_number
FROM section s
WHERE s.year = 2018
GROUP BY s.building, s.room_number
ORDER BY COUNT(*) ASC
LIMIT 1;
```

A-Z building	A-Z room_number	
Taylor	3128	

Tenemos que buscar los cursos que no tengan sección asociada. Para ello como tenemos que mostrar la información de los cursos pero filtrandola con los datos de las secciones realizamos un left join manteniendo toda la información en cursos. Una vez hecho esto nos quedamos con las secciones que no tengan ningún curso asociado "is null".

Para saber cual Aula es la menos utilizada es el equivalente a que aula aparece menos en la base de datos. Para ello agrupamos por los atributos que defines el aula y con "Count(*)" contamos las veces que aparece cada una. Mostramos por orden ascendente y únicamente uno para saber la menos usada.

6) PREGUNTA (1 punto): crea una consulta donde aparezcan los profesores y el número de asignaturas que imparten cada uno ordenados de mayor a menor número de asignaturas.

```
SELECT i.ID, i.name, COUNT(t.course_id) AS num_asignaturas
FROM instructor i
LEFT JOIN teaches t ON i.ID = t.ID
GROUP BY i.ID, i.name
ORDER BY num_asignaturas DESC;
```

A-Z ID	A-Z name	123 num_asignaturas	
10101	Srinivasan	3	
83821	Brandt	3	
45565	Katz	2	
76766	Crick	2	
12121	Wu	1	
15151	Mozart	1	
22222	Einstein	1	
32343	El Said	1	
98345	Kim	1	
33456	Gold	0	
58583	Califieri	0	
76543	Singh	0	

Para mostrar todos los profesores y el numero de asignaturas que imparten (aunque no impartan ninguna), Seleccionamos todos los profesores (Left join) y relacionamos por el ID de los profesores. Contamos los cursos que imparte cada uno "Count (t.course_id)" (se hace sobre la tabla que relaciona la entidad de los profesores con la de la asignatura) agrupando la información por el ID y el nombre del profesor (Contaríamos las veces que aparece su nombre y su id con un curso distinto)

7) PREGUNTA (1 punto): inspecciona la tabla *employees* de la base de datos *classicmodels*. Crea una consulta donde haya una columna con el nombre y apellido (concatenados) de los empleados y otra columna con el nombre y apellido (concatenada también) donde aparezca el jefe a quién reporta.

```
mysql> select * from employees;
```

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL	President
1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1	1002	VP Sales
1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1002	VP Marketing
1088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6	1056	Sales Manager (APAC)
1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4	1056	Sale Manager (EMEA)
1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1056	Sales Manager (NA)
1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com	1	1143	Sales Rep
1166	Thompson	Leslie	x4065	lthompson@classicmodelcars.com	1	1143	Sales Rep
1188	Firrelli	Julie	x2173	jfirrelli@classicmodelcars.com	2	1143	Sales Rep
1216	Patterson	Steve	x4334	spatterson@classicmodelcars.com	2	1143	Sales Rep
1286	Tseng	Foon Yue	x2248	ftseng@classicmodelcars.com	3	1143	Sales Rep
1323	Vanauf	George	x4102	gvanauf@classicmodelcars.com	3	1143	Sales Rep
1337	Bondur	Loui	x6493	lbondur@classicmodelcars.com	4	1102	Sales Rep
1370	Hernandez	Gerard	x2028	ghernande@classicmodelcars.com	4	1102	Sales Rep
1401	Castillo	Pamela	x2759	pcastillo@classicmodelcars.com	4	1102	Sales Rep
1501	Bott	Larry	x2311	lbott@classicmodelcars.com	7	1102	Sales Rep
1504	Jones	Barry	x102	bjones@classicmodelcars.com	7	1102	Sales Rep
1611	Fixter	Andy	x101	afixter@classicmodelcars.com	6	1088	Sales Rep
1612	Marsh	Peter	x102	pmarsh@classicmodelcars.com	6	1088	Sales Rep
1619	King	Tom	x103	tking@classicmodelcars.com	6	1088	Sales Rep
1621	Nishi	Mami	x101	mnishi@classicmodelcars.com	5	1056	Sales Rep
1625	Kato	Yoshimi	x102	ykato@classicmodelcars.com	5	1621	Sales Rep
1702	Gerard	Martin	x2312	mgerard@classicmodelcars.com	4	1102	Sales Rep

23 rows in set (0.31 sec)

```
SELECT
    CONCAT(e.firstName, ' ', e.lastName) AS empleado,
    CONCAT(m.firstName, ' ', m.lastName) AS jefe
FROM employees e
LEFT JOIN employees m ON e.reportsTo = m.employeeNumber;
```

A-z empleado	A-z jefe	
Diane Murphy	[NULL]	
Mary Patterson	Diane Murphy	
Jeff Hirrelli	Diane Murphy	
William Patterson	Mary Patterson	
Gerard Bondur	Mary Patterson	
Anthony Bow	Mary Patterson	
Leslie Jennings	Anthony Bow	
Leslie Thompson	Anthony Bow	
Julie Firrelli	Anthony Bow	
Steve Patterson	Anthony Bow	
Foon Yue Tseng	Anthony Bow	
George Vanauf	Anthony Bow	
Loui Bondur	Gerard Bondur	
Gerard Hernandez	Gerard Bondur	
Pamela Castillo	Gerard Bondur	
Larry Bott	Gerard Bondur	
Barry Jones	Gerard Bondur	
Andy Fixter	William Patterso	
Peter Marsh	William Patterso	
Tom King	William Patterso	
Mami Nishi	Mary Patterson	
Yoshimi Kato	Mami Nishi	
Martin Gerard	Gerard Bondur	

Agrupamos la información de dos columnas distintas de una misma tupla con la función concat, añadiendo un alias para reflejar la información. Se realiza un join recursivo que muestre todos los empleados pues en una misma tabla se encuentra el empleado y el jefe asociado. De esta forma buscamos las tuplas que cumplan las condiciones. La primera petición a employee sería para determinar el empleado y en la segunda determinaríamos el jefe. De aquí que tengamos que buscar una coincidencia entre el id del jefe que tenga un empleado asociado, entre todo los empleados registrados para así determinar su jefe.

La primera employees alias e, segunda employees alias m.

8) PREGUNTA (1 punto): ya conocemos algunas formas de importar datos en una tabla dentro de una base de datos a través de herramientas y funcionalidades del *dump*, basadas generalmente en sentencias SQL INSERT.

Se pide investigar una forma alternativa de realizar esta misma operación, es decir, importar datos de una tabla a una base de datos, pero en este caso siendo una tabla en formato .csv sin utilizar código SQL INSERT.

Dada la siguiente tabla ***asignatura*** con los datos que se especifican, transfórmala a un fichero en formato .csv (*asignatura.csv*) y crea una tabla (CREATE TABLE) llamada Asignatura en cualquier base de datos de tu sistema e importa los datos de la tabla *asignatura.csv* a la tabla Asignatura recién creada en tu SGBD.

Explica detalladamente todos los pasos seguidos para realizar este ejercicio y demostrando con ficheros, pantallazos, etc., todas las evidencias que justifiquen que has seguido el proceso de forma válida. **Prueba con una consulta SQL** que los datos de la tabla .csv se han importado correctamente a la correspondiente tabla de la base de datos en tu SGBD.

Ayuda: puedes utilizar el tutorial <https://www.mysqltutorial.org/import-csv-file-mysql-table/> de MySQL para realizar la investigación.

asignatura

ID	asignatura	semestre	creditos
1	Int BBDD	2	6
2	Sist. Oper.	1	6
3	Lab. Programacion	2	4,5

```
mysql> SELECT @@secure_file_priv
-> ;
+-----+
| @@secure_file_priv |
+-----+
| /var/lib/mysql-files/ |
+-----+
1 row in set (0,00 sec)

mysql>
```

```
set global local_infile = 1;

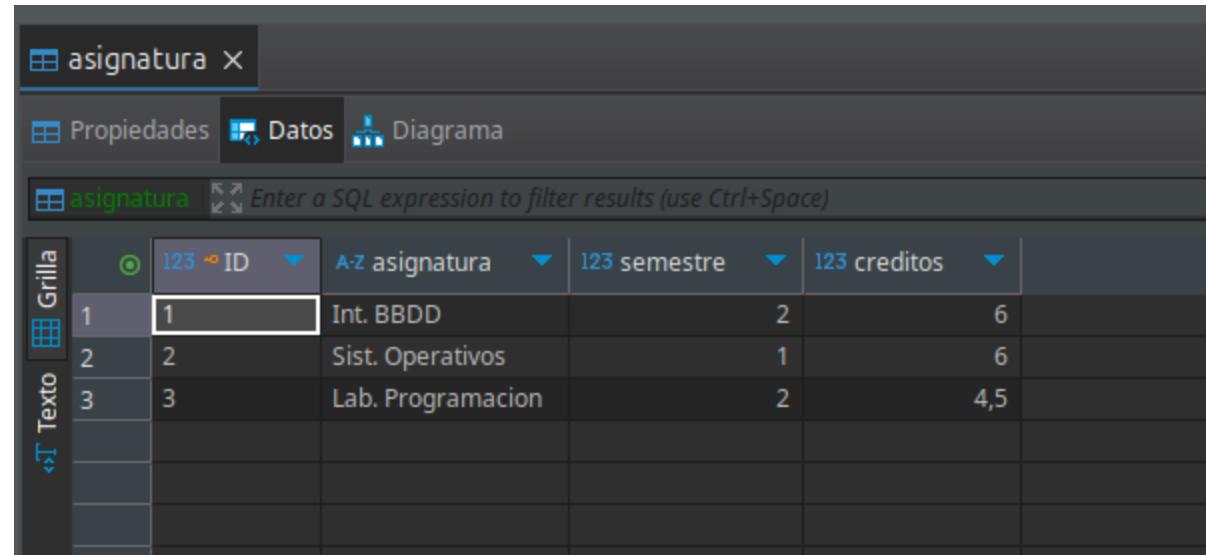
use classicmodels;

drop table if exists asignatura;

create table asignatura (
    ID int,
    asignatura varchar(50),
    semestre int,
    credits float,
    primary key (ID)
);

load data infile '/var/lib/mysql-files/asignatura.csv'
into table asignatura
fields terminated by ';'
enclosed by '"'
lines terminated by '\n'
ignore 1 rows;

select * from asignatura;
```



	123 ID	A-Z asignatura	123 semestre	123 credits
1	1	Int. BBDD	2	6
2	2	Sist. Operativos	1	6
3	3	Lab. Programacion	2	4,5

```
ID;Asignatura;Semestre;Creditos
1;Int. BBDD;2;6
2;Sist. Operativos;1;6
3;Lab. Programacion;2;4.5
```

```
mysql> SHOW TABLES;
```

```
+-----+
| Tables_in_classicmodels |
+-----+
| asignatura               |
| customers                |
| employees                |
| offices                  |
| orderdetails              |
| orders                   |
| payments                 |
| productlines              |
| products                  |
| vista                    |
+-----+
10 rows in set (0,00 sec)
```

```
mysql> SELECT * FROM asignatura;
```

```
+-----+-----+-----+-----+
| ID | asignatura      | semestre | credits |
+-----+-----+-----+-----+
| 1  | Int. BBDD       | 2        | 6       |
| 2  | Sist. Operativos | 1        | 6       |
| 3  | Lab. Programacion | 2        | 4.5     |
+-----+-----+-----+-----+
3 rows in set (0,01 sec)
```

La lógica del comando es:

Eliminar la tabla "asignatura" si esta ya existe en classicmodels.

Crear la tabla definiendo sus campos, el tipo de dato de cada campo y estableciendo uno de ellos como PK

Volcar el contenido del CSV en la tabla definiendo como esta estructurado este contenido; los campos estan separados por ";", el contenido por "'" y cada tupla o línea termina con "\n".

En el resto de los comandos comprobamos que todo se haya realizado bien.

9) PREGUNTA (1 punto): se pide mostrar en una misma tabla, y a través de una sola consulta, el top 5 de clientes que han realizado más pagos. En la tabla tendrán que estar el número de cliente, su nombre y el número de pagos que ha realizado cada uno (BBDD **classicmodels**)

Ayuda: hay que trabajar con las tablas *payments* y *customers*.

```
SELECT
    c.customerNumber,
    c.customerName,
    COUNT(p.checkNumber) AS cantidad_pagos
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber
GROUP BY c.customerNumber, c.customerName
ORDER BY cantidad_pagos DESC
LIMIT 5;
```

123 customerNumber	A-z customerName	123 cantidad_pagos	
141	Euro+ Shopping Channel	13	
124	Mini Gifts Distributors Ltd.	9	
114	Australian Collectors, Co.	4	
128	Blauer See Auto, Co.	4	
121	Baane Mini Imports	4	

Realizamos un innerJoin para obtener los clientes que han realizado algún pago. Sobre estos agrupamos por lo que define un cliente (su nombre y su código) y calculamos la cantidad de pagos que han realizado con "Count(p.checkNumber)" (Contar cuantas veces aparece la información del cliente con un checkNumber distinto). Ordenámos de mayor a menor con "Desc" y mostramos únicamente 5 con "Limit 5".

10) PREGUNTA (1 punto): de cara al próximo cuatrimestre, la empresa ClassicModels pretende aplicar una serie de descuentos extra para sus clientes más fieles. La cuantía de los descuentos se establece en función de las distintas líneas de producto, de manera que para cada tipo se aplicará el porcentaje de descuento siguiente: coches (Classic Cars → '5%'), aeroplanos (Planes → '2%'), motocicletas (Motorcycles → '4%'), embarcaciones (Ships → '3%') y resto un 6%.

Crea **una** sola consulta, en la que a partir del nombre de un producto se muestre una sola tabla, su línea de producto y el descuento correspondiente, por ejemplo, como en la **Figura10**, es decir, para el productName HMS Bounty (BBDD **clasicmodels**).

Ayuda: tienes que trabajar con la tabla **products**.

productName	productline	descuento
HMS Bounty	Ships	3%

Figura10


```
SELECT
  p.productName,
  p.productLine,
  CASE
    WHEN productLine = 'Classic Cars' THEN '5%'
    WHEN productLine = 'Planes' THEN '2%'
    WHEN productLine = 'Motorcycles' THEN '4%'
    WHEN productLine = 'Ships' THEN '3%'
    ELSE '6%'
  END AS descuento
FROM products p
WHERE p.productName = 'HMS Bounty';
```

A-z productName ▼	A-z productLine ▼	A-z descuento ▼	
HMS Bounty	Ships	3%	

La información de "descuento" sigue la misma lógica que unos if's anidados, de forma que en función del valor de "productLine" se le asigna un descuento al producto, así como pide el ejercicio. Toda esta operación solo se usa para los productos cuyo nombre sea "HMS Bounty" como dicta la instrucción "WHERE".



 Calle Playa de Liencres, 2 bis
(entrada por calle Rozabella)
Parque Europa Empresarial
Edificio Madrid
28290 Las Rozas, Madrid

 900 373 379  info@u-tad.com

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

